

PoisaPay

TECHNICAL DESIGN DOCUMENT · COMPLETE EDITION

System & Feature Design

Bangladesh-Focused · Custodial · Multi-Chain Crypto Wallet

A whole-system technical design for PoisaPay — a custodial multi-chain wallet (Ethereum · Tron · BNB Chain) for the Bangladesh market. **Part I** covers architecture, custody, ledger, money movement, data, APIs, security, compliance and infrastructure. **Part II** covers the full consumer feature set: cards (virtual & physical), local-currency / fiat accounts, send money & remittance, exchange, rewards, and crypto-backed credit. Key operations are numbered step-by-step flows.

Version	1.1 (Draft) — for technical review / sign-off
Stack	Laravel 12 · PHP 8.4 · PostgreSQL · Redis · Horizon · Reverb · Livewire 3 · Filament · Docker/Coolify · Nginx · Cloudflare · S3
Custody	HD (BIP32/39/44) · Hot / Cold split · M-of-N multisig
Companion	Implementation Guide Volumes I-II + Full Project Roadmap

Original design. Contains no proprietary code, branding, APIs, or UI from any existing company.

PoisaPay — Technical Design Document (TDD)

Document Control

Field	Value
Product	PoisaPay — Bangladesh-focused custodial multi-chain crypto wallet
Document	Technical Design Document (TDD)
Version	1.0 (Draft)
Status	For technical review / sign-off
Audience	Engineering, Security, DevOps, Compliance, Product
Scope	Whole-system design. Deep implementation for the financial core & blockchain/treasury layers lives in the companion Implementation Guide (Volumes I-II); Volumes III-VI are planned.
Non-goals	Not a business plan, not legal advice, not marketing/UI spec. Contains no proprietary code, branding, or APIs from any third party — original design only.

How to read this document. It is written to be read in order. Sections 1–3 set context and architecture; 4–6 are the money-critical designs (custody, ledger, movement); 7–11 cover data, APIs, security, compliance, and infrastructure; 12–15 cover NFRs, delivery, and open questions. Key operations (deposit, withdrawal, sweep, transfer) are written as **numbered step-by-step flows** so they can be implemented and tested literally step by step.

1. Introduction

1.1 Purpose

Define the technical design of PoisaPay: a custodial wallet that lets users in the Bangladesh market hold, receive, send, and (later) exchange crypto assets across Ethereum, Tron, and BNB Smart Chain, with all keys held and operated by the platform. The document exists so that engineering, security, and compliance can review and agree the design **before** large-scale implementation, and so that each subsystem has an unambiguous target.

1.2 Scope

In scope: custody (HD + hot/cold + multisig), a double-entry ledger, deposit/withdrawal/sweep money movement, blockchain monitoring, internal transfers, merchant payments and QR, exchange/swap, KYC/AML/risk, admin operations, REST API + webhooks, and the infrastructure to run and scale it.

Out of scope (this version): native mobile apps, card issuance, fiat on/off-ramp mechanics, and multi-currency beyond the launch assets — all tracked as future work (§13, Phase 10).

1.3 Definitions

Term	Meaning
Custodial	The platform holds users' private keys; user balances are ledger entries backed by pooled on-chain funds.
HD wallet	Hierarchical-deterministic (BIP32/39/44) key tree derived from one seed.
xpub / xpriv	Extended public / private key. Public derivation lets us generate addresses without private keys.
Deposit address	A per-user on-chain address for receiving; a transient holding pen, later swept.
Sweep	Moving funds out of deposit addresses into the pooled hot wallet.
Hot / Cold wallet	Online operational treasury / offline bulk-custody treasury.
Ledger account	A balance bucket in the double-entry ledger (a user "wallet" is a ledger account).
Base units	Smallest on-chain unit (wei/sun); all on-chain amounts stored as base units.
Reconciliation	Continuously proving $\sum \text{user liabilities} \leq \sum \text{on-chain funds we control}$.

1.4 References

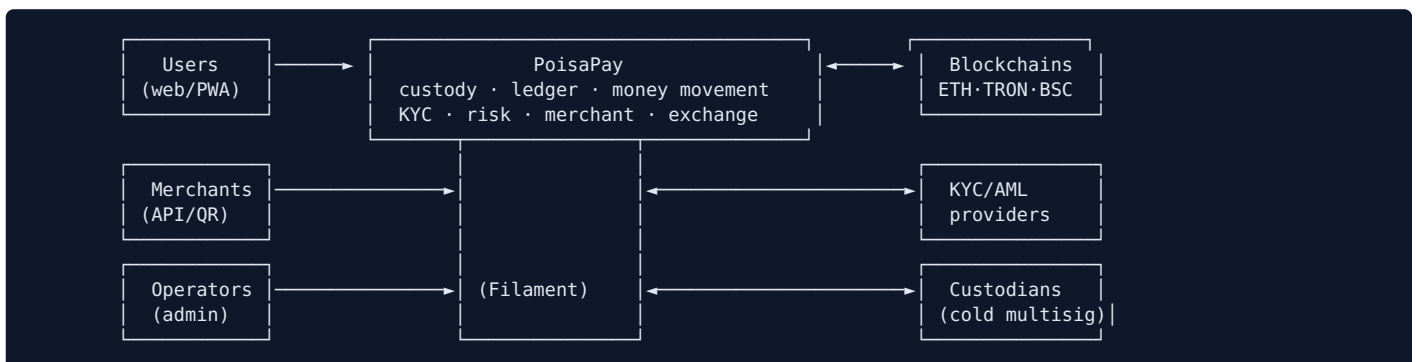
Companion Implementation Guide — Volume I (Foundations & Financial Core), Volume II (Blockchain & Treasury Operations), and the Full Project Roadmap. This TDD summarises and depends on the decisions made there; where it says "see Volume X" the exhaustive detail lives there.

2. System Overview

2.1 Product vision

PoisaPay is a custodial, multi-chain wallet for the Bangladesh market. The custodial choice is deliberate: it lets ordinary users transact without managing keys, at the cost of the platform bearing full custody responsibility. That single decision drives the entire design toward two obsessions — **an auditable ledger** and **isolated key custody**.

2.2 Context diagram



2.3 Key design decisions (load-bearing, with rationale)

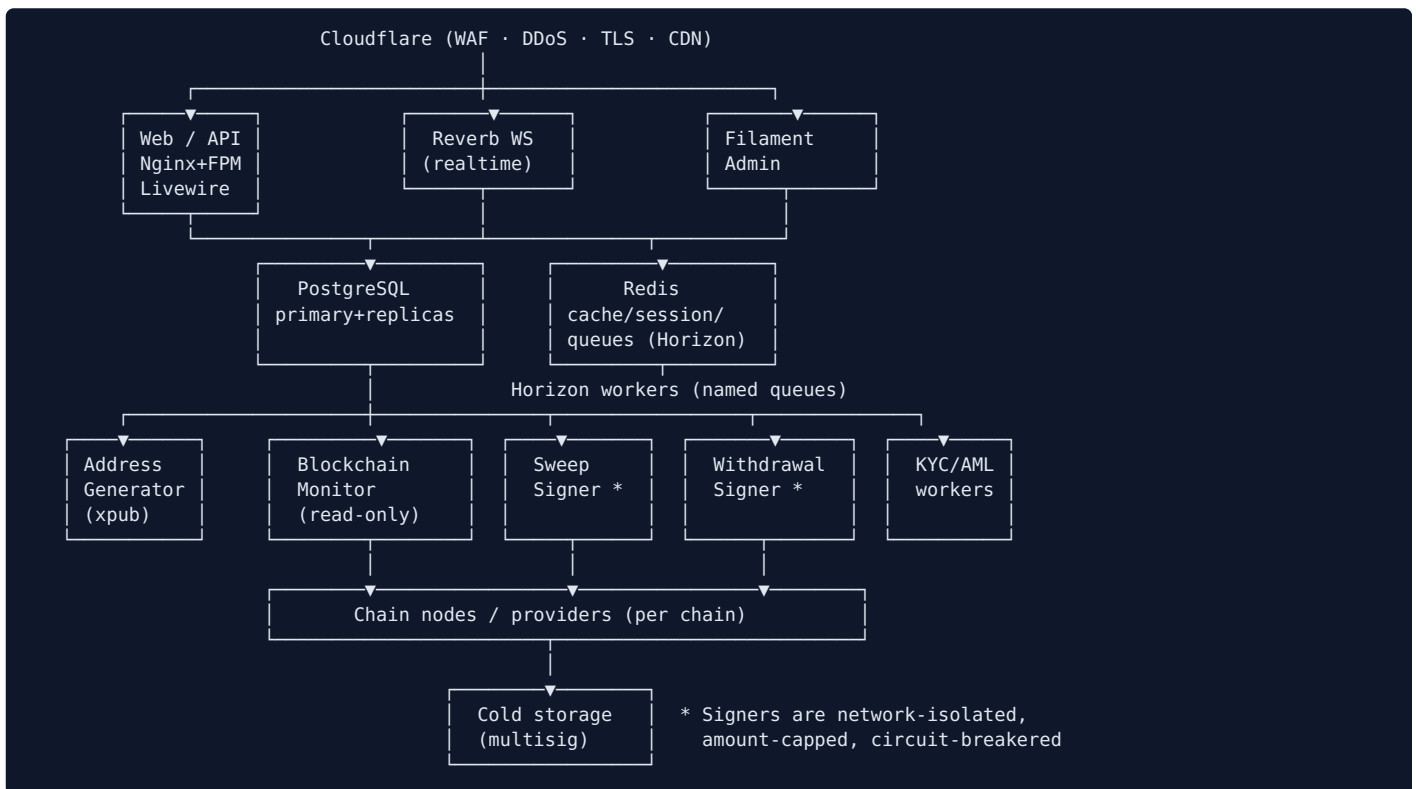
#	Decision	Rationale
D1	Ledger is source of truth for user balances; chain is source of truth for aggregate treasury.	Custodial model. User balance must not depend on where coins physically sit or on sweep timing.
D2	Double-entry, append-only ledger. No mutation; corrections are reversing entries.	Auditability by construction; "who changed this balance and why" is always answerable.
D3	Money never stored as float. On-chain amounts <code>NUMERIC(78,0)</code> base units; ledger <code>NUMERIC(38,0)</code> minor units; arithmetic via <code>brick/math</code> .	Exactness is non-negotiable in a financial ledger.
D4	Address derivation is public; signing is private. Address generator holds only xpub.	A full compromise of the online zone leaks no funds directly.
D5	Every external side effect is idempotent (keyed).	Queues deliver at-least-once; retries must not double-credit or double-spend.
D6	Confirmations are per-asset data, not constants; credit only after N confs + canonical re-check.	Reorg safety.
D7	Tron-first, then EVM.	Cheapest sweeps + dominant USDT rail in-market; forces the token path early.
D8	Gate-driven delivery (no real money before KYC/AML + audit gates).	Custodial risk; see §13.

2.4 Assumptions & constraints

- Launch assets: ETH (native), BNB (native), TRX (native), and USDT on each of the three chains (six asset-on-chain rows).
- Regulatory posture for custodial crypto in Bangladesh is restrictive and is treated as a **first-class design input** (see §10.4), not an afterthought.
- The team operates dedicated RPC/node access per chain with provider failover.
- A custodian roster exists for cold multisig (M-of-N).

3. Architecture Design

3.1 High-level architecture



3.2 Trust zones (the security spine)

Three isolation zones, and the design's most important boundary is the gap between the first two:

1. **Online zone** — web/API, admin, Address Generator, Blockchain Monitor. Holds **no private keys**. A full breach here lets an attacker read balances and generate addresses, not move funds.
2. **Signing zone** — Sweep & Withdrawal signers. Network-isolated, holds hot keys (decrypted only in memory per operation), amount-capped and rate-limited with a circuit breaker. Blast radius = the small hot float.
3. **Cold zone** — offline. Master seed + cold multisig keys. Never online; cold→hot is human-gated M-of-N.

3.3 Component catalog

Component	Responsibility	Reads	Writes	Zone
Web/API (Livewire)	User & merchant interface, REST API	ledger, wallets	requests, transfers	Online
Filament Admin	Operator console (approve, support, report)	everything	approvals, config	Online
Address Generator	Derive deposit addresses from xpub	xpubs, counters	deposit_addresses	Online
Blockchain Monitor	Detect deposits, track confs, reorgs	chain nodes	onchain_txs, deposits	Online
Ledger Service	Post balanced entries (only path value moves)	balances	journal entries/lines	Online
Sweep Signer	Sign & broadcast sweeps	signing jobs	chain, broadcast_attempts	Signing
Withdrawal Signer	Sign & broadcast withdrawals	signing jobs	chain, broadcast_attempts	Signing
KYC/AML workers	Verify identity, screen, monitor	providers	kyc, screening results	Online
Reconciliation	Prove solvency vs chain	chain, ledger	reconciliation runs	Online

3.4 Technology choices

Laravel 12 / PHP 8.4 (service+repository, Form Requests, Actions, DTOs, PSR-12); PostgreSQL (exact `NUMERIC`, partial unique indexes, `FOR UPDATE`, advisory locks); Redis (queues, cache, sessions, address set, nonce/rate

locks); Horizon (named queues with isolated concurrency); Reverb (realtime balance push); Livewire 3 + Alpine + Tailwind (UI); Filament (admin); Docker + Coolify + Nginx + Cloudflare + S3 (infra).

4. Custody Design

4.1 HD wallet hierarchy

One BIP39 seed (24 words, generated air-gapped) derives everything. Account level uses **hardened** derivation per chain; address level uses **non-hardened** so deposit addresses can be derived from an account **xpub** without private keys (see Volume I §8).

Chain	Path template	Coin type	Encoding
Ethereum	<code>m/44'/60'/0'/0/{i}</code>	60	EIP-55 (<code>0x...</code>)
BNB Smart Chain	<code>m/44'/60'/1'/0/{i}</code>	60 (EVM)	EIP-55 (<code>0x...</code>)
Tron	<code>m/44'/195'/0'/0/{i}</code>	195	Base58Check (<code>T...</code>)

4.2 Address generation

The Address Generator receives only the account **xpub** and derives `/0/{index}` via public derivation. It cannot produce a private key. The derivation index is allocated atomically from a per-chain monotonic counter under a row lock (no `MAX+1`). See Volume I §8.3–8.5.

4.3 Hot / Cold / Multisig

- **Hot wallet:** minimal operational float (floor/target/cap policy). Funds withdrawals and sweep gas. Its balance is the maximum online blast radius.
- **Cold wallet:** offline bulk custody, watch-only monitored via xpub. Cold→hot is manual + M-of-N + air-gapped.
- **Multisig:** EVM Safe-style contract; Tron native weighted-permission multisig. Proposal → M approvals → broadcast, with per-signer no-double-approve. See Volume II §16–18.

4.4 Key ceremony

Seed generated air-gapped, split via Shamir across custodians, xpubs registered online, hot keys provisioned into the secrets vault. Executed at delivery Phase 4 with a signed, archived runbook.

5. Ledger Design

5.1 Double-entry model

Every value movement is a **balanced journal entry** of ≥ 2 lines where $\sum \text{debits} = \sum \text{credits}$, enforced in code and by a deferred DB trigger. Accounts include user wallets and system accounts (`treasury:hot|cold|pending` , `fee:income` , `gas:expense` , `liability:user-funds`). See Volume I §9.

5.2 Balance components

Each user wallet exposes `available` , `locked` (reserved), and `total = available + locked` . Locking is itself a balanced transfer `user:available → user:locked` — no special-case field mutation.

5.3 States & idempotency

Business events move `pending → completed | failed | reversed` . Lines are never edited; a correction is a new reversing entry linked by `reverses_entry_id` . Every entry carries an `idempotency_key` (e.g. `deposit:{txhash}:{vout}`); a retry collapses to a no-op.

5.4 Reconciliation

Scheduled per asset: assert `on-chain controlled ≈ ledger treasury` (within an in-flight tolerance) and `ledger treasury ≥ ledger liability` (solvency). Per-address completeness checks recover missed deposits after an outage. Drift or insolvency pages on-call and can freeze operations. See Volume II §12.9.

6. Money Movement Design (step-by-step flows)

6.1 Deposit flow

```

Step 1 User requests a deposit address (or reuses one) for asset-on-chain.
Step 2 API allocates index (atomic) → Address Generator derives address → store + add to Redis watch set.
Step 3 User sends funds on-chain to the address.
Step 4 Monitor detects the tx (native scan or token Transfer log); records onchain_tx + deposit (detected).
Step 5 On each new block, confirmations recomputed; on reorg, canonical re-check.
Step 6 At min_confirmations + still-canonical → post entry (treasury:pending → user:available). Deposit=credited.
Step 7 Reverb pushes the new balance to the user in realtime.
Step 8 (Async, later) Sweep policy may move the coins to the hot wallet – independent of the user's balance.

```

6.2 Sweep flow

```

Step 1 Deposit-address token balance crosses per-asset threshold (or scheduled).
Step 2 Fund gas: EVM → gas-station sends native coin to the address; Tron → delegate energy/bandwidth.
Step 3 Sweep Signer builds + signs the token-move tx (nonce-managed on EVM; expiry-managed on Tron).
Step 4 Broadcast (idempotent by nonce/context); monitor confirms.
Step 5 Ledger posts sweep.settle (treasury:hot → treasury:pending) + gas expense. Sweep=swept.

```

Note: sweep never touches the user's wallet; it is treasury-internal.

6.3 Withdrawal flow

```

Step 1 User submits withdrawal (asset, amount, to_address); validated (min/max, address format).
Step 2 Risk/AML checks run (screening, velocity). Above auto-threshold or flagged → manual admin approval.
Step 3 RESERVE FIRST: lock funds (user:available → user:locked). Balance can't be double-spent now.
Step 4 Withdrawal Signer signs + broadcasts from hot wallet (nonce-managed, idempotent by id).
Step 5 Monitor confirms at N confs.
Step 6 Ledger settles: debit user:locked, credit treasury:out, debit fee. Withdrawal=completed.
Step 7 If cancelled/failed BEFORE broadcast → reverse the lock (user:locked → user:available).

```

Invariant: **reserve before you sign** — never touch the chain before the ledger has locked the funds.

6.4 Internal transfer

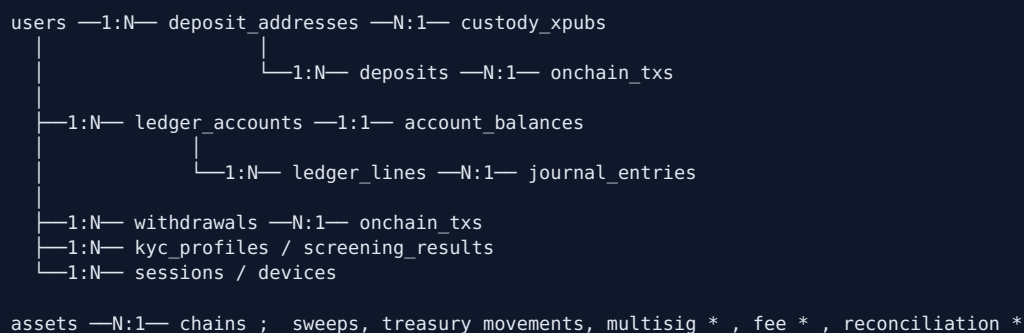
User→user transfer is a pure ledger operation (no chain): a single balanced entry `sender:available → recipient:available`, atomic, idempotent, instantaneous, zero network fee. This is a major UX and cost advantage of the custodial model.

6.5 Chain-specific notes

- **ERC-20 / BEP-20:** detect via `Transfer` event logs (contract + topic0), match `to` in-app against the Redis set. EVM nonce + EIP-1559 gas.
- **TRC-20:** detect via Tron event logs; hex↔base58 conversion; solidity-block confirmations; energy delegation for cheap sweeps.
- Respect per-asset `decimals` (USDT=6, native=18); store base units verbatim.

7. Data Design

7.1 ER overview



7.2 Core tables

Full DDL (columns, types, indexes, constraints, business rules) is in Volume I §39 and Volume II. Design-level summary of the money-critical set:

Table	Purpose	Critical constraint
<code>chains</code> , <code>assets</code>	Registry, decimals, min_confirmations	partial unique for native assets
<code>custody_xpubs</code>	Account xpubs (public only)	CHECK: never an xpriv
<code>deposit_addresses</code>	Per-user receive addresses	unique (chain,address) & (xpub,index)
<code>journal_entries</code>	Balanced value events	unique idempotency_key
<code>ledger_lines</code>	Debit/credit lines	deferred balanced-entry trigger
<code>account_balances</code>	Materialised balances	locked under <code>FOR UPDATE</code> on post
<code>deposits</code>	Incoming funds	unique onchain_tx (no double-credit)
<code>withdrawals</code>	Outgoing funds	unique idempotency_key
<code>sweeps</code> , <code>broadcast_attempts</code>	Treasury moves, retries	idempotent by nonce/context
<code>multisig_*</code> , <code>reconciliation_*</code>	Cold approvals, solvency	no-double-approve; drift alerts

7.3 Money-handling policy

On-chain: `NUMERIC(78,0)` base units. Ledger: `NUMERIC(38,0)` minor units. No float/double anywhere. Display formatting applies `decimals` only at the edge.

7.4 Indexing & partitioning

Hot-path indexes on `deposit_addresses(address)`, `ledger_lines(account_id, created_at)`, status columns on deposits/withdrawals/sweeps. High-volume append tables (`ledger_lines`, `onchain_txs`) are partition candidates by time as volume grows. GIN/trigram for any admin `ilike` search. See Volume V (planned) for the full index strategy.

8. API Design

8.1 Conventions

- **Versioned:** all routes under `/v1`. **Auth:** Bearer token (Sanctum) for users/merchants; admin is Filament session, not public API.
- **Idempotency:** mutating endpoints accept an `Idempotency-Key` header; server dedupes.
- **Errors:** consistent envelope `{ error: { code, message, details } }` with stable machine codes.
- **Rate limits:** per-endpoint, dual-window (per-account + per-IP), returned via `X-RateLimit-*` headers. Sensitive endpoints (auth, withdrawal) are tighter.

- **Validation:** Form Requests; never trust client input.

8.2 Endpoint catalog (design-level)

Group	Method & path	Auth	Purpose
Auth	POST /v1/auth/register	none	Create account
Auth	POST /v1/auth/login	none	Login (returns 2FA challenge if enabled)
Auth	POST /v1/auth/2fa/verify	partial	Complete 2FA
Auth	POST /v1/auth/otp/{request,verify}	partial	SMS/email OTP (daily-capped)
Auth	GET/DELETE /v1/sessions, GET /v1/devices	user	Session & device management
KYC	POST /v1/kyc/submit, GET /v1/kyc/status	user	Submit documents, check tier
Reference	GET /v1/assets, GET /v1/chains	user	Supported assets/chains
Wallet	GET /v1/wallets, GET /v1/wallets/{asset}	user	Balances (available/locked/total)
Deposit	POST /v1/deposit-addresses	user	Get/create a deposit address
Deposit	GET /v1/deposits, GET /v1/deposits/{id}	user	Deposit history/status
Withdrawal	POST /v1/withdrawals	user (2FA)	Request withdrawal (Idempotency-Key)
Withdrawal	GET /v1/withdrawals, GET /v1/withdrawals/{id}	user	History/status
Transfer	POST /v1/transfers	user (2FA)	Internal user→user transfer
Exchange	POST /v1/swaps/quote, POST /v1/swaps	user	Quote & execute swap
Merchant	POST /v1/merchant/invoices, GET /v1/merchant/invoices/{id}	merchant	Create/query invoice
Merchant	POST /v1/merchant/payments, QR payload	merchant/ user	Pay invoice / QR

Each endpoint's full spec (request, response, error codes, rate limit, emitted events) is elaborated in Volume IV (planned); this catalog fixes the surface and conventions.

8.3 Webhooks & events

Outbound webhooks (merchant/integrator) with HMAC signature, retry with backoff, and delivery log: `deposit.confirmed`, `withdrawal.completed`, `withdrawal.failed`, `swap.completed`, `invoice.paid`. Internal domain events drive queued listeners (notifications, webhooks, reconciliation hooks).

9. Security Design

9.1 Trust boundaries

Per §3.2: online (no keys) / signing (isolated, capped) / cold (offline, multisig). This is the primary control.

9.2 Authentication & authorisation

2FA (TOTP), OTP (SMS/email, daily-capped), device verification, server-side session management with revocation, and RBAC/permission system for both users and operators (Spatie-style). Admin actions are permission-gated and audit-logged.

9.3 Encryption & secrets

TLS in transit (Cloudflare + origin). Hot keys encrypted at rest via envelope encryption (data key wrapped by KMS/HSM), decrypted only in signer memory per operation. Secrets (RPC keys, KMS grants, DB creds) in a vault, injected at runtime, never in repo or image. DB `CHECK`s refuse anything key-shaped in address/hot tables.

9.4 Signer controls

Amount caps + rate limits at the infrastructure level; a **circuit breaker** trips if rolling outflow exceeds the expected envelope, freezing signing and paging on-call. Even a fully compromised signer cannot exceed the small hot float before a human is alerted.

9.5 Rate limiting, fraud, audit

Dual-window rate limiting (per-account + per-IP) with CAPTCHA escalation on auth; withdrawal risk scoring and daily outflow circuit breakers; immutable, tamper-evident audit trail for all privileged and money-moving actions. Full treatment in Volume V (planned).

10. Compliance & Risk Design

10.1 KYC (tiered)

Identity verification (NID/passport + liveness/document check via provider) with **verification tiers** that gate limits: unverified (view-only / no withdrawal), basic, and full (higher limits). Onboarding cannot progress to real-money actions without the required tier.

10.2 AML

Sanctions/PEP screening at onboarding and per-withdrawal; transaction monitoring with velocity/pattern rules; a SAR (suspicious activity report) workflow for compliance operators. Screening results are recorded and auditable.

10.3 Risk engine

Per-withdrawal risk scoring feeding the auto-approve vs. manual-review decision; per-user and platform-wide daily outflow circuit breakers; anomaly flags surfaced to operators. Thresholds are configurable data.

10.4 Bangladesh regulatory posture (design input)

The regulatory status of custodial crypto services in Bangladesh is restrictive and subject to change. This TDD treats compliance as a **hard gate** (no real-money onboarding before KYC/AML is live — §13) and isolates compliance so the posture can be tightened without re-architecting. Specific licensing, permitted-activity, and reporting obligations are a matter for qualified legal counsel and are an explicit dependency, not an engineering assumption.

11. Infrastructure & Deployment Design

11.1 Container topology

Docker services orchestrated by Coolify: `app` (PHP-FPM), `nginx`, `postgres` (+ replicas), `redis-cache`, `redis-queue`, `horizon` workers (multiple, by queue), `reverb`, and isolated `signer` services on separate hosts/networks. Multi-stage images, pinned versions, `opcache.validate_timestamps=0`, config/route/view cache on deploy.

11.2 Environments

`local` (Docker), `staging` (testnet keys, production-like), `production` (mainnet). Testnet-first: every chain flow is proven on staging/testnet before mainnet keys exist.

11.3 Networking & zone isolation

Cloudflare (WAF/DDoS/TLS) → Nginx → app. Signer hosts have **no inbound internet**; they pull signing jobs and push signed txs. Cold zone is fully offline. Wildcard TLS via DNS-01 for tenant/subdomain needs (per prior Caddy/Cloudflare experience; Nginx used here per stack).

11.4 Scaling

PostgreSQL read replicas + PgBouncer (transaction pooling); Redis split (cache `allkeys-lru`; queue/session `noeviction` + AOF); Horizon per-queue concurrency so deposit-crediting, sweeping, and withdrawal-signing never starve each other; RPC provider failover; edge caching for public/reference endpoints. See Volume VI (planned).

11.5 Observability, backup, DR

Centralised logs/metrics/traces; alerting on reconciliation drift, signer circuit breaker, queue depth, provider health. Postgres backup + point-in-time recovery; documented DR runbook with tested RTO/RPO. On-call rotation with runbooks.

12. Non-Functional Requirements

Attribute	Target (proposed)	Notes
Availability	99.9% for user API	HA Postgres/Redis, multi-worker Horizon
API latency	p95 < 300 ms (read), < 800 ms (write)	excludes on-chain confirmation time
Deposit detection	< 1 block behind tip	monitor per chain
Durability	zero fund/ledger loss	append-only ledger + reconciliation + backups
Security	no open critical/high post-audit	Phase 7 gate
Compliance	KYC/AML on 100% of money-moving users	Phase 5 gate
Scalability	define concrete peak-TPS & concurrent-session target	"10M users" must be quantified before load testing (open question O1)
Maintainability	PSR-12, PHPStan L8, Pest tests	CI-enforced

13. Delivery Plan (summary)

Gate-driven, testnet-first (full detail in the Full Project Roadmap):

P0 Foundations → P1 Ledger+Custody → P2 Tron (testnet) → P3 EVM (testnet) → P4 Cold+Multisig+Key ceremony → P5 KYC/AML ♦ money gate → P6 Product+Admin → P7 External audit ♦ audit gate → P8 Scale/HA/DR → P9 Staged launch → P10 Future

Two hard gates dominate: **no real-money onboarding before P5**, and **no public launch before P7**. Compliance/Legal and the external auditor are the long poles — engaged from P0 and P5 respectively.

14. Risks & Open Questions

Risks (owner in parentheses): signing-key compromise → catastrophic (Security); ledger drift → insolvency (Backend); regulatory block (Compliance); RPC dependency (DevOps); sweep gas economics (Blockchain); reorg double-credit (Blockchain). Mitigations are designed in above and tracked in the roadmap risk register.

Open questions - **O1** — Concrete scalability target: quantify "10M users" as peak TPS + concurrent sessions before load testing (Phase 8). - **O2** — Final launch asset list and per-asset confirmation thresholds (compliance + treasury sign-off). - **O3** — KYC/AML provider selection for the Bangladesh market (compliance). - **O4** — Custodian roster and M-of-N parameters for cold multisig. - **O5** — Legal permitted-activity determination (counsel) — gates P5. - **O6** — Fee schedule and whether network fees are passed to users or absorbed.

15. Appendix — Decision Log

ID	Decision	Status
D1-D8	See §2.3	Accepted
A1	PostgreSQL over MySQL (exact NUMERIC, partial indexes, locking)	Accepted
A2	Tron-first chain rollout	Accepted
A3	Reserve-before-sign on withdrawals	Accepted
A4	Internal transfers as pure ledger ops (no chain)	Accepted
A5	Signer isolation + amount cap + circuit breaker	Accepted
A6	Compliance as hard gate before real money	Accepted

End of PoisaPay Technical Design Document v1.0 (Draft).

PART II

Consumer Feature Design

Cards · Local Currency · Send Money · Exchange · Rewards · Credit

PoisaPay — Consumer Feature Design (Part II)

Extends the PoisaPay TDD with the consumer-facing feature set of a RedotPay-class product: **cards** (virtual + physical), **local-currency / fiat accounts**, **send money & payouts**, **exchange / conversion**, **rewards & referral**, and **crypto-backed credit**. Feature sections are numbered **F1-F7** to sit cleanly alongside the TDD's §1-15. Same house conventions (append-only ledger, `NUMERIC` money, idempotency, trust-zone isolation).

F0. What changes, and the honest dependencies

These features do **not** change the financial core — they are new *products* built on the same ledger. But three of them (cards, fiat ramps, credit) introduce **licensed-partner and regulatory dependencies** that must be stated plainly, because they are architectural facts, not footnotes:

Feature	Hard external dependency	Why it can't be self-built
Cards	Issuer / BIN-sponsor + card network (Visa/Mastercard) program , and PCI-DSS scope	You cannot mint network-branded cards yourself. PoisaPay is a <i>program manager / processor integrator</i> that provisions cards through a licensed issuer's API and authorises each transaction via a real-time webhook.
Fiat / local currency	PSP / bank / mobile-wallet rails (e.g. bank transfer, bKash/Nagad) + regulatory permission for crypto↔fiat	Moving BDT and converting crypto↔fiat is the most regulated activity in the platform; it requires partners and a licence/permission.
Crypto-backed credit	Lending permission + liquidation/market infrastructure	Lending is separately regulated; collateral liquidation needs deep, reliable market access.

⚠ **Regulatory reality for Bangladesh.** Card issuance, fiat on/off-ramps, and lending each compound the \$10.4 regulatory gate. This document designs the *technical* system so it is ready when permission and partners exist; it does **not** assert that these activities are currently permitted. Each is a hard dependency on qualified counsel and licensed partners — treat F2/F5/F7 as "designed, gated on legal + partner," not "buildable today."

The design principle that makes all of this tractable: **it's all just the ledger**. A fiat balance is a ledger account whose asset is a currency. A card hold is a lock. A payout is a debit. A reward is a credit. Cards and fiat don't need a second money system — they need new asset types and new orchestration around the same double-entry core.

F1. Multi-Currency & Local (Fiat) Accounts

F1.1 Design

Generalise the asset model so the ledger holds **fiat currencies** (BDT, USD, EUR...) exactly as it holds crypto. A fiat "account" is just a `ledger_account` whose asset is a currency with `decimals = 2`. Everything the ledger already guarantees — double-entry, idempotency, available/locked — applies unchanged.

- **Base display currency:** the app shows a user's total in one chosen currency (e.g. BDT or USD), computed from live FX; the *stored* balances remain per-asset and exact.
- **Payment priority:** the user orders which assets are spent first (e.g. USDT → BDT → BTC). Stablecoin-first is recommended to minimise volatility exposure.

F1.2 Schema (delta)

```
ALTER TABLE assets ADD COLUMN kind VARCHAR(8) NOT NULL DEFAULT 'crypto'; -- 'crypto' | 'fiat'
ALTER TABLE assets ADD COLUMN currency_code CHAR(3); -- 'BDT','USD' for fiat
-- fiat assets: kind='fiat', decimals=2, contract_address/chain_id nullable/synthetic

CREATE TABLE user_spending_priority (
  user_id      UUID          NOT NULL REFERENCES users(id),
  position     SMALLINT      NOT NULL, -- 1 = spend first
  asset_id     SMALLINT      NOT NULL REFERENCES assets(id),
  PRIMARY KEY (user_id, asset_id),
  CONSTRAINT uq_priority_pos UNIQUE (user_id, position)
);

CREATE TABLE ramp_orders (
  id           UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id      UUID          NOT NULL REFERENCES users(id),
  direction    VARCHAR(4)    NOT NULL, -- 'on' (fiat->balance) | 'off' (balance->fiat out)
  rail         VARCHAR(24)    NOT NULL, -- 'bank_transfer' | 'mobile_wallet' | 'card_topup'
  fiat_asset_id SMALLINT      NOT NULL REFERENCES assets(id),
  fiat_amount  NUMERIC(38,0) NOT NULL, -- minor units (paisa)
  provider_ref VARCHAR(128), -- PSP/bank reference
  status       VARCHAR(24)    NOT NULL DEFAULT 'pending', -- pending|confirmed|credited|failed
  entry_id     UUID          REFERENCES journal_entries(id),
  created_at   TIMESTAMPTZ    NOT NULL DEFAULT now(),
  CONSTRAINT uq_ramp_provider UNIQUE (rail, provider_ref) -- idempotent against PSP callback
);
```

F1.3 On-ramp flow (fund fiat balance)

```
Step 1 User initiates on-ramp: amount + rail (bank / mobile wallet).
Step 2 Create ramp_order (pending). Present pay instructions / redirect to PSP.
Step 3 User pays via the rail; PSP sends a confirmation callback (HMAC-verified, idempotent by provider_ref).
Step 4 On confirmed funds → post ledger entry (ramp:clearing → user:available:BDT). ramp_order=credited.
Step 5 User's BDT balance updates in realtime (Reverb).
```

Off-ramp is the mirror: lock fiat → PSP payout to the user's bank/wallet → on payout confirmation, settle.

△ On/off-ramp settlement is asynchronous and partner-paced; never credit before the PSP confirms cleared funds (same discipline as on-chain confirmations).

F2. Exchange & Conversion Engine

F2.1 Design

One engine serves three needs: crypto↔crypto swaps, crypto↔fiat conversion, and the **just-in-time (JIT) conversion** that card spending depends on (F3). It is quote-driven with an explicit spread:

- **Quote:** given `(from_asset, to_asset, amount)`, return a rate (from a rate source), a spread (bps), the resulting amount, and an **expiry** (quotes are short-lived — seconds — because rates move).
- **Execute:** re-validate the quote hasn't expired, lock the `from` amount, post a balanced conversion entry, credit the `to` amount. Spread accrues to `fx:spread_income`.

F2.2 Schema

```
CREATE TABLE fx_quotes (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID          REFERENCES users(id),
  from_asset_id SMALLINT    NOT NULL REFERENCES assets(id),
  to_asset_id SMALLINT    NOT NULL REFERENCES assets(id),
  from_amount NUMERIC(78,0) NOT NULL,
  to_amount   NUMERIC(78,0) NOT NULL,
  rate        NUMERIC(38,18) NOT NULL,
  spread_bps  INT           NOT NULL,
  source      VARCHAR(32)   NOT NULL,      -- rate provider
  expires_at  TIMESTAMPTZ   NOT NULL,      -- typically now()+ a few seconds
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now()
);

CREATE TABLE conversions (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID          NOT NULL REFERENCES users(id),
  quote_id    UUID          NOT NULL REFERENCES fx_quotes(id),
  context     VARCHAR(16)   NOT NULL,      -- 'swap' | 'ramp' | 'card_settle'
  entry_id    UUID          REFERENCES journal_entries(id),
  idempotency_key VARCHAR(128) NOT NULL,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),
  CONSTRAINT uq_conversion_idem UNIQUE (idempotency_key)
);
```

F2.3 Volatility / FX risk

JIT conversion means PoisaPay carries price risk between a card authorisation hold and its settlement. Mitigations designed in: prefer stablecoins in spending priority; hold a small buffer above the quoted amount; and run a treasury hedging/spread policy so the aggregate spread income covers realised slippage. This is a treasury-operations concern (Volume II family), surfaced here because cards create the exposure.

F3. Card System (the "card generator")

The headline feature. **PoisaPay does not issue cards itself** — it integrates a licensed **issuer / BIN-sponsor + processor** and a card network (Visa/Mastercard). "Card generator" = provision a virtual/physical card via the partner API, then authorise each spend in real time against the user's balances with JIT conversion.

F3.1 PCI scope — stay out of it by design

PoisaPay never stores or touches the PAN/CVV. It holds only a **network token / issuer card reference**. Card reveal and Apple/Google/Samsung Pay provisioning happen through the partner's PCI-compliant vault (iframe/SDK), so PoisaPay's own systems stay out of PCI-DSS cardholder-data scope. This is the single most important card-security decision.

F3.2 Card lifecycle

Issue (virtual instant / physical shipped) → active → (freeze/unfreeze, set limits, rotate) → spend → (dispute/refund) → close. Cards require a higher KYC tier (\$10.1). Types: **virtual** (online + mobile-wallet) and **physical** (POS + ATM + tap-to-pay).

F3.3 The authorisation flow (latency-critical)

This is the crown-jewel sequence. The card network gives only a few seconds to approve, so the auth path must be **fast** — Redis-cached balances and pre-warmed quotes, no slow queries.


```

Step 1 User taps/swipes; network → issuer-processor → real-time AUTH webhook to PoisaPay
(card token, amount, settlement currency, MCC, merchant).
Step 2 Resolve card → user; check card status, KYC tier, per-card + velocity limits, risk score.
Step 3 Pick funding asset by the user's spending priority; get a JIT crypto→fiat quote (incl. spread).
Step 4 Sufficient available balance? If not (or risk/limit fails) → DECLINE (respond fast).
Step 5 Place HOLD: ledger locks the crypto equivalent (user:available → user:card_hold).
Step 6 Respond APPROVE to the webhook within the network timeout. (p99 budget: ~2s – hard NFR.)
Step 7 Later, network sends CLEARING/SETTLEMENT (amount may differ: tips, partial, FX).
Step 8 Settle: convert held crypto→fiat (realise via F2), debit user:card_hold,
credit card_program:settlement, post card fee + fx spread. Release any over-hold.
Step 9 Refund: merchant refund → credit back to user balance per policy. Dispute → F3.6 workflow.

```

F3.4 Card ledger accounts

`user:card_hold:{asset}` (authorisation holds), `card_program:settlement:{fiat}` (owed to issuer), `fee:card`, `fx:spread_income`. A hold is a lock; settlement is a lock→settlement transfer plus a conversion — all standard double-entry.

F3.5 Schema

```

CREATE TABLE cards (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID          NOT NULL REFERENCES users(id),
  program     VARCHAR(24)   NOT NULL,          -- issuer program id
  type        VARCHAR(8)    NOT NULL,          -- 'virtual' | 'physical'
  network     VARCHAR(12)   NOT NULL,          -- 'visa' | 'mastercard'
  issuer_card_ref VARCHAR(128) NOT NULL,        -- partner token; NEVER the PAN
  last4       CHAR(4),
  status       VARCHAR(16)  NOT NULL DEFAULT 'inactive', -- inactive|active|frozen|closed
  daily_limit  NUMERIC(38,0),                  -- minor units, settlement currency
  per_tx_limit NUMERIC(38,0),
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_issuer_card UNIQUE (issuer_card_ref),
  -- defence in depth: refuse anything that looks like a raw PAN in the token column
  CONSTRAINT ck_no_pan CHECK (
    length(issuer_card_ref) > 19
    OR (issuer_card_ref NOT LIKE '4%' AND issuer_card_ref NOT LIKE '5%')
  )
);

CREATE TABLE card_authorizations (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  card_id     UUID          NOT NULL REFERENCES cards(id),
  network_auth_id VARCHAR(64) NOT NULL,        -- from the network; idempotency key
  amount      NUMERIC(38,0) NOT NULL,          -- settlement currency minor units
  currency_code CHAR(3)     NOT NULL,
  mcc         CHAR(4),
  merchant    VARCHAR(128),
  funding_asset_id SMALLINT REFERENCES assets(id),
  held_amount  NUMERIC(78,0),                  -- crypto base units locked
  quote_id     UUID          REFERENCES fx_quotes(id),
  status       VARCHAR(16)  NOT NULL DEFAULT 'approved', -- approved|declined|settled|reversed|expired
  hold_entry_id UUID          REFERENCES journal_entries(id),
  settle_entry_id UUID       REFERENCES journal_entries(id),
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
  CONSTRAINT uq_network_auth UNIQUE (network_auth_id) -- re-sent auths never double-hold
);

CREATE INDEX ix_cardauth_card ON card_authorizations (card_id, created_at);

CREATE TABLE card_disputes (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  authorization_id UUID      NOT NULL REFERENCES card_authorizations(id),
  reason      VARCHAR(48)   NOT NULL,
  status      VARCHAR(16)   NOT NULL DEFAULT 'open', -- open|represented|won|lost
  amount      NUMERIC(38,0) NOT NULL,
  entry_id     UUID          REFERENCES journal_entries(id),
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT now()
);

```

F3.6 Disputes & chargebacks

Card programs carry chargeback liability. A dispute reverses the settlement in the ledger (a reversing entry, never a mutation), tracks representment with the issuer, and records win/loss. Chargeback losses hit a `card_program:loss` account and are monitored — high dispute rates are both a cost and a risk signal.

F3.7 Card APIs

Method & path	Auth	Purpose
POST /v1/cards	user (KYC-gated)	Issue virtual/physical card via partner
GET /v1/cards, GET /v1/cards/{id}	user	List / detail
POST /v1/cards/{id}/freeze · /unfreeze	user (2FA)	Freeze controls
PUT /v1/cards/{id}/limits	user (2FA)	Set daily / per-tx limits
POST /v1/cards/{id}/reveal	user (2FA)	Reveal via partner PCI iframe (no PAN through us)
POST /v1/cards/{id}/wallet-token	user	Apple/Google/Samsung Pay provisioning
GET /v1/cards/{id}/transactions	user	Card history
PUT /v1/spending-priority	user	Order funding assets
POST /webhooks/card-auth	partner (HMAC)	Inbound real-time authorisation (F3.3)

F4. Send Money, Transfers & Payouts

Three distinct products, one ledger:

F4.1 Internal P2P (instant, free)

Send to another PoisaPay user by handle (phone / email / username). Pure ledger op: one balanced entry `sender:available -> recipient:available`, atomic, idempotent, instant, zero network fee. Supports crypto *and* fiat assets. If the recipient handle isn't registered, hold as a claimable pending transfer with expiry.

F4.2 Fiat payout (off-platform)

Send fiat to an external bank account or mobile wallet — an off-ramp targeted at a third party. Lock funds → PSP payout → settle on confirmation. Same machinery as F1 off-ramp with an external beneficiary.

F4.3 Remittance (cross-border) — high value for the Bangladesh market

Cross-border remittance is a major use case here. Sender funds in crypto/fiat; recipient receives **BDT** to a local bank or mobile wallet. Designed as: source lock → cross-border settlement (via stablecoin rail internally) → local BDT payout via PSP. Compliance-heavy (both ends KYC'd, AML screened, purpose codes) — gated behind the money gate and remittance-specific permissions.

F4.4 Schema (delta)

```
CREATE TABLE transfers (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  sender_id   UUID          NOT NULL REFERENCES users(id),
  recipient_id UUID          REFERENCES users(id),          -- NULL until a claimable is claimed
  recipient_handle VARCHAR(128),          -- phone/email/username if off-platform
  asset_id    SMALLINT      NOT NULL REFERENCES assets(id),
  amount      NUMERIC(78,0) NOT NULL,
  kind        VARCHAR(16)   NOT NULL,          -- 'internal' | 'payout' | 'remittance'
  status      VARCHAR(16)   NOT NULL DEFAULT 'completed', -- pending|claimable|completed|reversed
  entry_id    UUID          REFERENCES journal_entries(id),
  idempotency_key VARCHAR(128) NOT NULL,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),
  CONSTRAINT uq_transfer_idem UNIQUE (idempotency_key)
);
CREATE INDEX ix_transfers_sender ON transfers (sender_id, created_at);
```

F4.5 Send APIs

POST /v1/send (P2P by handle), POST /v1/payouts (fiat payout), POST /v1/remittances (cross-border), GET /v1/transfers / GET /v1/transfers/{id}. All mutating routes take an `Idempotency-Key` and require 2FA above a threshold.

F5. Rewards & Referral

F5.1 Design

Rewards are ledger credits into `rewards:*` accounts, claimable/spendable per policy. Three mechanisms: welcome bonus, referral rewards (referrer + referee), and optional card cashback / daily stablecoin rewards. Keeping rewards in the ledger means they reconcile like any other balance and can't be double-granted (idempotency on the grant).

F5.2 Schema

```
CREATE TABLE referrals (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  referrer_id UUID          NOT NULL REFERENCES users(id),
  referee_id  UUID          NOT NULL REFERENCES users(id),
  code        VARCHAR(24)  NOT NULL,
  status      VARCHAR(16)  NOT NULL DEFAULT 'pending', -- pending|qualified|rewarded
  reward_entry_id UUID     REFERENCES journal_entries(id),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_referral_pair UNIQUE (referrer_id, referee_id),
  CONSTRAINT ck_no_self_referral CHECK (referrer_id <> referee_id)
);
```

Rewards grants are idempotent (`reward:{type}:{user}:{period}`) and anti-abuse rules gate qualification (referee must complete KYC + a qualifying action) to prevent farming.

F6. Crypto-Backed Credit (design note)

A borrow-against-collateral product: the user pledges crypto as collateral and draws a credit line (in fiat or stablecoin) without selling, repaying flexibly. Core mechanics: collateral lock, loan-to-value (LTV) ratio, interest/fee accrual, margin monitoring, and **liquidation** if collateral value falls below a maintenance threshold.

- Ledger: `user:collateral:locked`, `credit:principal`, `credit:accrued_fee`, and a liquidation path that sells collateral (via F2) to cover the outstanding line.
- Risk: continuous mark-to-market of collateral; automated margin calls and liquidation; conservative LTV.

⚠ Lending is **separately regulated** and introduces liquidation/market-risk machinery beyond the wallet core. This is a later-phase product, gated on lending permission and robust market access — designed here for completeness, not for near-term build.

F7. How the features change the delivery plan

These slot into the roadmap *after* the money gate, because every one of them touches real user funds and most carry partner/regulatory dependencies:

Feature	Roadmap placement	Added gate/dependency
F1 Fiat accounts + on/off-ramp	Phase 6 (Product)	PSP/bank rails + fiat permission
F2 Exchange/conversion	Phase 6	rate source + treasury spread policy
F4 Send / payouts / remittance	Phase 6	remittance permission (cross-border)
F3 Cards	Phase 6-7 (after audit)	issuer/BIN + network + PCI — the longest partner lead time; start partner selection at Phase 0 alongside compliance
F5 Rewards / referral	Phase 6	anti-abuse rules
F7 Credit	Phase 10 (future)	lending permission + liquidation infra

New hard NFR from cards: card authorisation must respond within the network timeout — **p99 < ~2s on the auth path** — which constrains that path to Redis-cached balances and pre-warmed quotes. This is now a first-class performance requirement (add to §12, ref O1).

Updated open questions: issuer/BIN-sponsor + card-network partner selection; PSP/rail partners for BDT; rate-source provider; remittance corridors and permissions; lending permission for F6. All gate on legal + partner, and the card partner is the longest pole — engage it early.

End of PoisaPay Consumer Feature Design (Part II). Read together with the TDD §1–15, this now covers the full RedotPay-class feature set: crypto, cards, local currency, send money, exchange, rewards, and credit.